

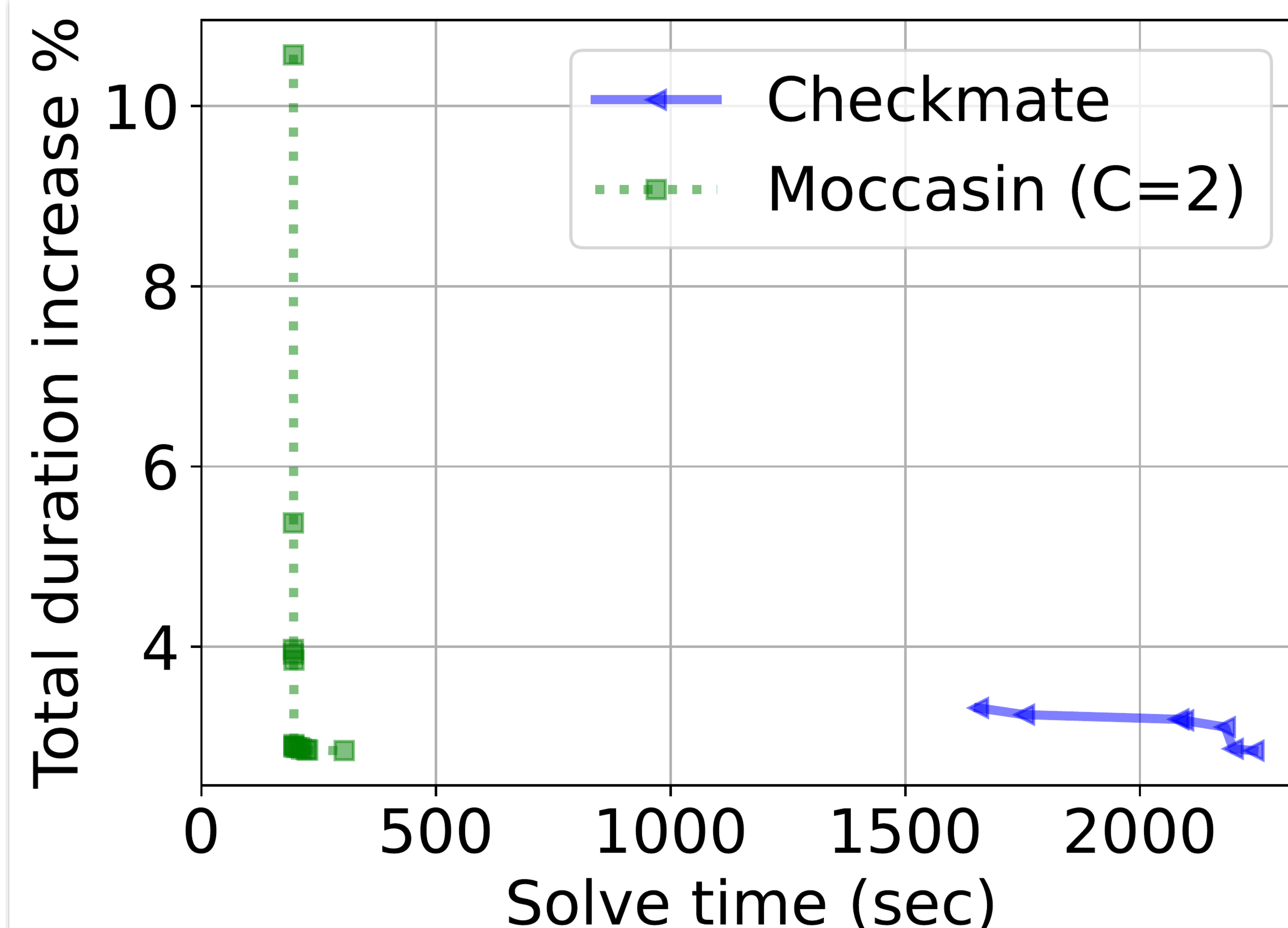
Problem

Given a compute graph and a fixed **memory budget**, decide which tensors to **keep** versus **recompute** so total execution time is minimized while peak memory stays within budget.

*The schedule search is **PSPACE-complete** — and memory is the primary limit for on-device training.*

Motivation

- **Checkmate** (prior SOTA) casts rematerialization as a MILP with $O(n^2)$ Boolean variables.
- That quadratic growth is a wall: large graphs **time out** or exhaust memory, and cannot scale past a few hundred nodes.

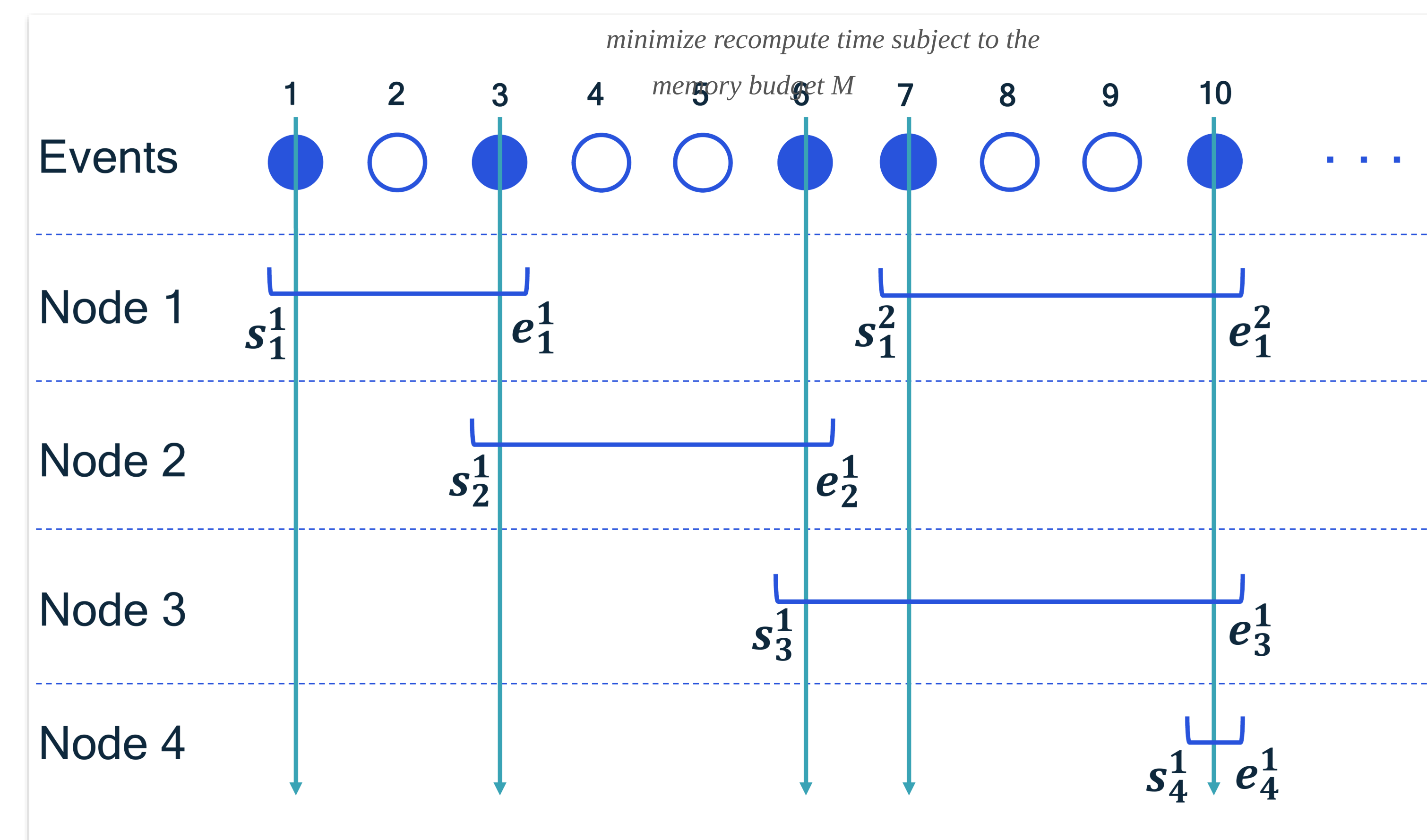


On a real graph ($n=442$, $m=1247$), Checkmate needs ~2000 s while Moccasin (green) converges near ~200 s.

Method

- Each node's tensor = up to C_v **retention intervals** (start s , end e , flag a).
- CP **cumulative** + **reservoir** constraints; solved by OR-Tools **CP-SAT** with $O(Cn)$ variables.

$$\min_{s,e,a} \sum_{v,i} w_v a_{iv} \quad \text{s.t.} \quad \sum_{v,i} m_v a_{iv} \leq M$$



Retention intervals: each node's tensor lifetime as a small set of [start, end] event intervals (s , e).

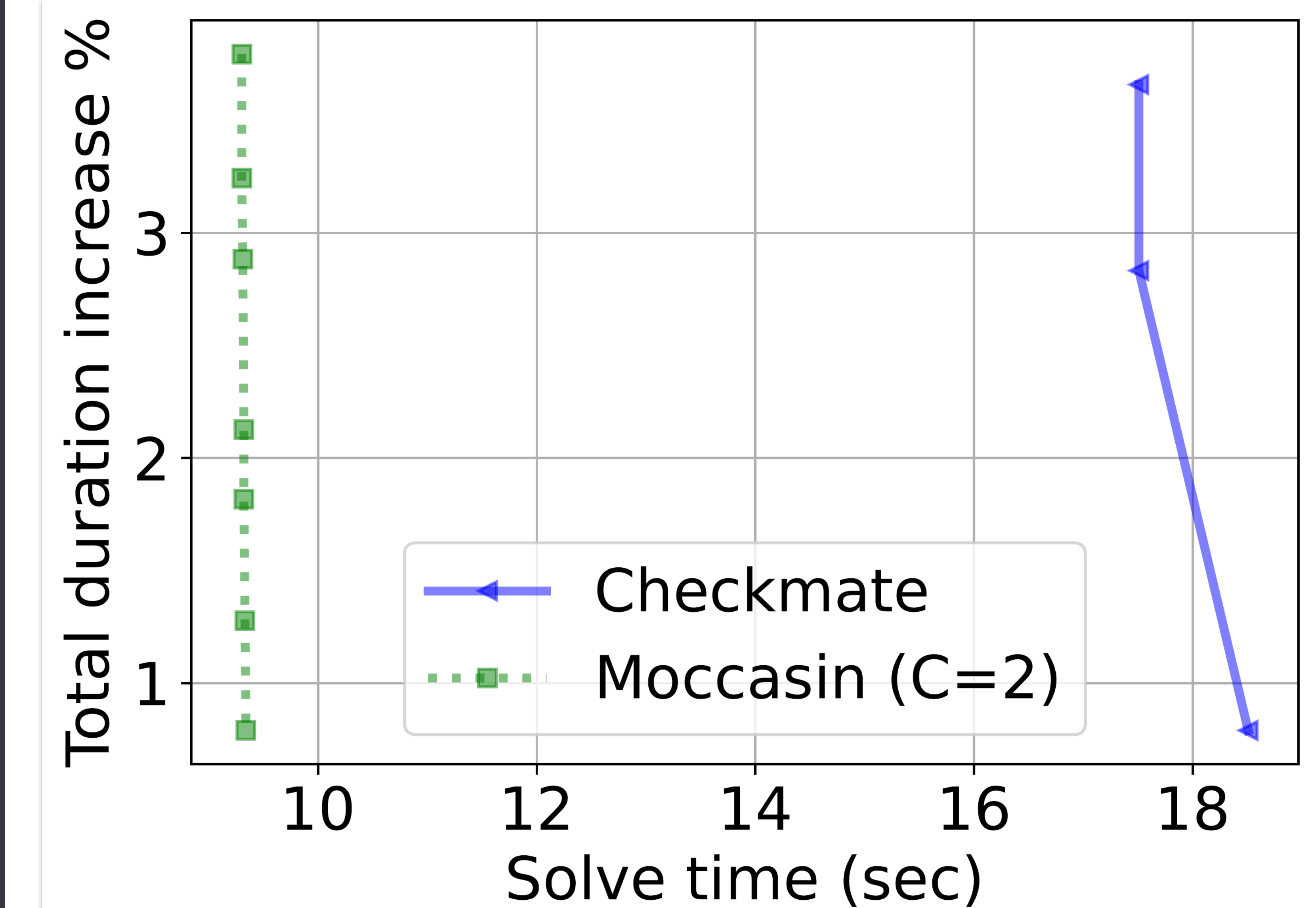
Takeaway

Reformulating rematerialization over retention intervals cuts discrete variables from quadratic to linear — solving far larger graphs up to 10× faster, overhead under 5%.

The variable count grows only **linearly** in graph size; the solved sequence is hardware-agnostic and runs unchanged on any CPU or GPU.

Key Results

METHOD	SOLVE TIME	DUR. INC.
Checkmate (MILP)	~2000 s / OOM	~3%
Moccasin (C=2)	~200 s / <1 h	<5%



Solve progress on random layered graphs: Moccasin (green, ~9.5 s) reaches a low-overhead solution far sooner than Checkmate (blue, ~18 s).

SO WHAT → On G3/G4 Checkmate returns no solution in 3 h and hits OOM; Moccasin finishes under an hour.

Headline Numbers

10×	$O(n)$ integer vars (vs $O(n^2)$)
faster solve on large graphs	<5% duration overhead
vs. Checkmate, up to an order of magnitude faster on the largest graphs tested	1000 nodes, $m=5875$ solved
	C=2 intervals per node